

Atomic Composite Mode - One Signature, All-or-Nothing

Status: LIVE on mainnet. A multi-step intent such as "swap 2 SUI to USDC then lend it on NAVI" can execute as **one Programmable Transaction Block (PTB), one wallet signature**, all-or-nothing - instead of two separate signed steps.

This is Dewlock's headline chaining capability and a direct expression of the moat: **the LLM only proposes the legs; a deterministic builder composes them and the Guardian re-derives and fail-closes before any signature is requested.**

1 - What the user does

1. Type a compound intent: swap 2 SUI to USDC then lend it on navi.
2. A **chain-plan card** appears with two steps and a toggle: "Run as 1 transaction (atomic)".
3. Click it -> a single tx-preview card renders with the full **flow map** (both legs, real protocol logos, estimated output).
4. Sign **once** in the wallet -> both legs settle together, or neither does.

The atomic toggle is offered only when the plan is **eligible**: exactly two steps, leg 0 = swap, leg 1 = lend_deposit on NAVI (recipe swap_lend_v1), and nothing has been signed yet.

2 - How one PTB is built (swap_lend_v1)

The composite builder (packages/sui/src/build-composite.ts -> buildLiveSwapLendPtb) assembles both legs into a single PTB whose swap-output coin feeds the lend input **structurally** - no wallet round-trip between legs:

Step	What happens	Why it matters
Coverage gate	assertSuiGasCoverage - total SUI >= swap amount + 0.05 SUI gas reserve, else InsufficientGasCoverageError	A shortfall is caught before the route fetch, surfaced as the same insufficient_gas gate as a normal swap - not a confusing "route unavailable" error
Route	Cetus aggregator findRouters (venues: CETUS + DEEPBOOK) -> best route + estimated output	Same engine as a normal swap, so multi-path routes compose cleanly

Swap leg	<code>routerSwap({ router, txb, inputCoin, slippage })</code> returns <code>targetCoin</code> (the swap-output coin as a PTB argument)	<code>routerSwap</code> (not <code>fastRouterSwap</code>) returns the coin instead of sending it to the wallet - that is what makes it composable
Link	split exactly <code>minAmountOut</code> (the slippage floor) from <code>targetCoin</code> -> <code>depositCoin</code>	Reproduces the proven NAVI deposit invariant (coin value == deposit amount); the floor is always <= the real output, so the split never aborts
Lend leg	<code>navi.depositCoinPTB(txb, USDC, depositCoin, ...)</code>	The swap output is deposited in the same PTB
Dust	<code>transferObjects([targetCoin], sender)</code> returns the small remainder above the floor	No coin object dangles; the remainder goes back to the user

Atomicity: a Move abort in *either* leg (slippage exceeded, NAVI deposit rejected, ...) reverts the **entire** PTB. Nothing executes on a partial failure.

Why Cetus, not Aftermath: the earlier builder used Aftermath's `addTransactionForCompleteTradeRoute`, which splits the gas-token SUI input during a deferred resolution step that aborts (MoveAbort 46001) for multi-path routes - so atomic *always* fell back. The Cetus aggregator returns the output coin directly and is the same route engine normal swaps already use.

3 - The Guardian re-verifies the whole composite

The builder produces bytes; it does **not** grant trust. `checkCompositeRecipe` (`packages/agent/src/guardian.ts`) re-derives four invariants on the **built PTB** before returning an `approvedDigest`:

1. **Closed-recipe registry** - only declared recipes (`swap_lend_v1`) are composable. No ad-hoc composition.
2. **Target multiset** - every MoveCall in the PTB must be in the recipe's `allowedTargets` (the Cetus aggregator swap calls + NAVI `entry_deposit` / `refresh_stake`), and **every leg must be present**. `allowSignatureMatch` admits the per-hop `<dex>::swap` calls by `module::function`.
3. **Coin-type linkage** - the swap output coin type must equal the lend input coin type (the structural proof the legs are connected).
4. **Delta/owner anti-leak + dual caps** - no third-party owner or balance delta, USD value within `TX_USD_CAP`, and net-SUI outflow within `NET_SUI_DELTA_CAP_MIST`.

Any failure -> **BLOCK** (no signature). The approved digest is bound to the exact bytes (**WYSIWYS**): if the bytes change after approval, signing is refused.

4 - The flow map - clear input -> output at every node

The composite tx-preview renders an explicit **You -> Cetus -> NAVI** chain (both the node graph and the row view), because the intermediate USDC nets to ~0 at the wallet and would otherwise be invisible in balance deltas:

```
[You] - 2 SUI -> [Cetus Aggregator - Swap -> ~6.2 USDC] - ~6.2 USDC -> [NAVI - Lending - deposit]
```

- **Real protocol logos** on each node (Cetus, NAVI, ...), resolved from the bundled brand assets.
- **Estimated output** from the live route shown on the swap node and on the edge into the lend node, so every node has a clear **in** (incoming edge) and **out** (estimated amount).
- The estimate is the aggregator's own route number (same value a normal swap preview shows); the **minimum** is the slippage floor that actually executes.

5 - Graceful fallback (funds and safety never affected)

If the composite cannot be built or the Guardian blocks it, atomic **degrades to the sequential step-by-step chain** - the user signs each step in order, and every Guardian check still applies. The fallback message is classified honestly:

- **Balance/gas shortfall** -> *"Not enough SUI to swap that amount and still cover network gas. Reduce the amount or top up."* (Falling back would hit the same shortfall, so it doesn't pretend to help.)
- **Route can't be composed** -> *"Atomic bundling isn't available for this swap route right now ... falling back to step-by-step; your funds and every Guardian check are unaffected."*

Sequential mode also auto-recovers from a transient stale-object error by rebuilding the step with fresh bytes (bounded retries; stale bytes are never re-sent, so a coin is never equivocated).

6 - Scope & limitations

- **v1 recipe:** `swap_lend_v1` only (SUI -> USDC -> NAVI deposit). The architecture is recipe-driven, so adding a recipe = declaring its legs + allowed targets in `composite-recipes.ts`.
- **Page refresh** loses an in-flight sequential chain (durable resume not yet implemented). The atomic path is a single signature, so it isn't affected mid-flight.
- The **estimate** is a dry-run figure; the guaranteed amount is the slippage-floor minimum.

7 - Why this is safe (the moat, restated)

- The LLM proposes **legs**, never bytes. The deterministic builder composes them; the Guardian re-derives the full PTB and fail-closes.
- **One signature, WYSIWYS** - the user signs the exact approved bytes; any change after approval is refused.
- **All-or-nothing** - a partial failure reverts everything.
- **Zero user-fund keys server-side** - Dewlock never holds spend authority; the wallet signs the single composed PTB.

Related: [system-architecture.md](#) (full Guardian gate list), [copilot-command-guide.md](#) (end-user command reference).